Свёрточные нейронные сети

От свёрточной арифметики до знаковых архитектур

Курс «Deep Learning»

2026

План лекции

- Мотивация: что не так с полносвязными слоями?
- Что такое свёртка?
- Свёрточная арифметика
- Базовые блоки CNN
- Pooling и его виды
- Специализированные свёртки
- Знаковые архитектуры:
  - LeNet, AlexNet, VGG
  - Inception, ResNet
  - DenseNet, MobileNet, EfficientNet

Мотивация: полносвязные слои неэффективны

Проблемы FC-слоёв для изображений:

- $224 \times 224 \times 3$ *цветное изображение* $\rightarrow$ 150 528 нейронов во входном слое
- Скрытый слой 1024 нейрона: **150 млн параметров** только на первом слое
- Нет инвариантности к сдвигу
- Нет локальности: каждый пиксель влияет на каждый нейрон

Идея CNN

Локальность: нейрон «видит» только небольшой участок изображения (receptive field).
Разделение весов: один и тот же фильтр применяется ко всем участкам изображения.

- В 10–100 раз меньше параметров
- Инвариантность к сдвигу
- Естественная иерархия признаков

Что такое свёртка?

1D свёртка (дискретная):

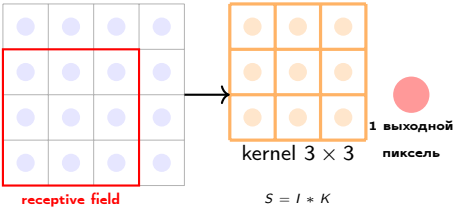
$$(f * g)[n] = \sum_{m=-\infty}^{\infty} f[m] \cdot g[n-m]$$

2D свёртка (кросс-корреляция, на практике):

$$S[i,j] = \sum_{u=-U}^{U} \sum_{v=-V}^{V} I[i+u, j+v] \cdot K[u+U, v+V]$$

Компоненты:

- I — вход (изображение, feature map)
- K — ядро (kernel, filter), размер $K_h \times K_w$
- S — выход (feature map)



Ядро «скользит» по изображению: скалярное произведение в каждом окне.

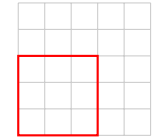
Свёрточная арифметика

Основная формула

$$O = \left\lfloor \frac{I + 2P - K}{S} \right\rfloor + 1$$

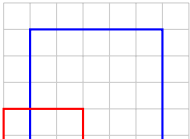
где I — размер входа, K — размер ядра, P — padding, S — stride, O — размер выхода.

Padding = 0, Stride = 1



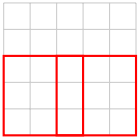
$5 \times 5 \to 3 \times 3$

Padding = 1, Stride = 1



$5 \times 5 \to 5 \times 5$

Padding = 0, Stride = 2



$5 \times 5 \to 2 \times 2$

Типы padding:

- **Valid** ($P = 0$): $O = (I - K)/S + 1$, сжимает
- **Same** ($P = \lfloor K/2 \rfloor$): $O = I/S$, сохраняет

Пример: $I = 32$, $K = 5$, $S = 1$:

- Valid: $O = 32 - 5 + 1 = 28$
- Same: $O = 32$ (с $P = 2$)

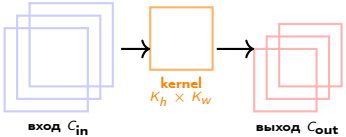
Многоканальные свёртки

Свёртка с несколькими каналами:
Вход: $H \times W \times C_{\text{in}}$
Ядро: $K_h \times K_w \times C_{\text{in}} \times C_{\text{out}}$
Выход: $H' \times W' \times C_{\text{out}}$

Каждый выходной канал — сумма свёрток по всем входным каналам:

$$S_j = \sum_{i=1}^{C_{\text{in}}} I_i * K_{ij}$$

Параметры: $K_h \cdot K_w \cdot C_{\text{in}} \cdot C_{\text{out}} + C_{\text{out}}$ bias



3×3, $C_{\text{in}} = 64$, $C_{\text{out}} = 128 \to 73\,728$ параметров

3D свёрточная арифметика

Полная формула для тензора

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} + 2P_h - K_h}{S_h} \right\rfloor + 1$$

$$W_{\text{out}} = \left\lfloor \frac{W_{\text{in}} + 2P_w - K_w}{S_w} \right\rfloor + 1$$

$$D_{\text{out}} = C_{\text{out}} \quad (\text{задаётся})$$

Количество параметров слоя:

$$\text{params} = K_h \cdot K_w \cdot C_{\text{in}} \cdot C_{\text{out}} + C_{\text{out}}$$

Размер receptive field:

$$RF_l = RF_{l-1} + (K_l - 1) \cdot \prod_{i=1}^{l-1} S_i$$

Примеры:

Слой	Вход C_{in}	Параметры
3×3, 64	3	$3 \cdot 3 \cdot 3 \cdot 64 + 64 = 1792$
5×5, 128	64	$5 \cdot 5 \cdot 64 \cdot 128 + 128$ $= 204\,928$
3×3, 512	512	$3 \cdot 3 \cdot 512 \cdot 512 + 512$ $= 2\,359\,808$

Pooling — субдискретизация

Max Pooling:

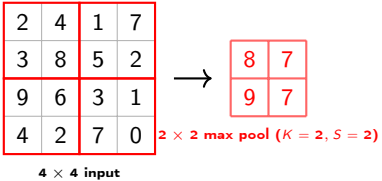
$$y_{ij} = \max_{u=1..K, v=1..K} x_{i \cdot S+u, j \cdot S+v}$$

- Инвариантность к малым сдвигам
- Выделяет наиболее активный признак
- Нет обучаемых параметров

Average Pooling:

$$y_{ij} = \frac{1}{K^2} \sum_{u,v} x_{i \cdot S+u, j \cdot S+v}$$

Более «гладкий» выход.
Global Pooling: pooling по всему пространству $\rightarrow$ 1×1.



Зачем нужен pooling: уменьшение размерности $\rightarrow$ меньше параметров $\rightarrow$ bigger receptive field без роста вычислений.

Транспонированная свёртка (Deconvolution)

Зачем: увеличение пространственного разрешения.
Используется в:

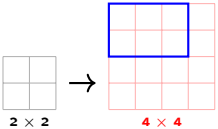
- Генеративных моделях (GAN, VAE)
- Сегментации (U-Net, DeepLab)
- Super-resolution

Идея: обратная операция к обычной свёртке. Если свёртка — «многие-к-одному» (many-to-one), то транспонированная — «один-ко-многим» (one-to-many).

Формула:

$$O = (I - 1) \cdot S + K - 2P$$

Пример: 2×2 input, 3×3 kernel, $S = 2$:



Важно: это НЕ обратная свёртка. Это свёртка с дробным шагом (fractionally-strided convolution).

Dilated (Atrous) Convolution

Идея: увеличиваем receptive field без увеличения числа параметров.

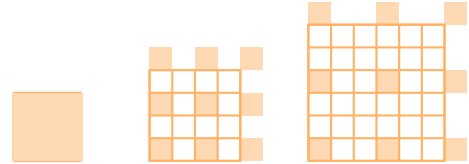
Формула:

$$K_{\text{effective}} = K + (K - 1) \cdot (d - 1)$$

где d — dilation rate.

Пример: 3×3 ядро:

- $d = 1$: $K_{\text{eff}} = 3$, RF $= 3 \times 3$
- $d = 2$: $K_{\text{eff}} = 5$, RF $= 5 \times 5$
- $d = 4$: $K_{\text{eff}} = 9$, RF $= 9 \times 9$



$d = 1$: 3×3 $d = 2$: 5×5 $d = 3$: 7×7

Одно и то же 3×3 ядро, но разный dilation. Параметров не прибавляется!

Depthwise Separable Convolution

Стандартная свёртка:

$$\text{params} = K^2 \cdot C_{\text{in}} \cdot C_{\text{out}}$$

Depthwise Separable (2 шага):
1. Depthwise: $K \times K$ свёртка для *каждого* канала отдельно:

$$\text{params}_{\text{dw}} = K^2 \cdot C_{\text{in}}$$

2. Pointwise (1×1)**:** линейная комбинация каналов:

$$\text{params}_{\text{pw}} = C_{\text{in}} \cdot C_{\text{out}}$$

Сравнение параметров:

	$3 \times 3,\ C_{\text{in}} = 64,\ C_{\text{out}} = 128$
Обычная	$3^2 \cdot 64 \cdot 128 = 73\,728$
Depthwise	$3^2 \cdot 64 = 576$
Pointwise	$64 \cdot 128 = 8\,192$
Итого	**8 768**
Ускорение	**8.4x**

Используется в MobileNet, Xception, EfficientNet.

1×1 Свёртка (Pointwise Convolution)

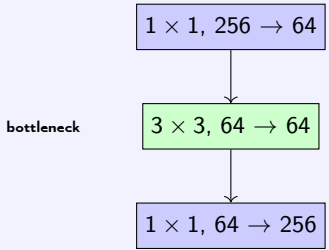
Что делает: линейная комбинация каналов без пространственного взаимодействия.

$$y_{ij}^{(k)} = \sum_{c=1}^{C_{\text{in}}} w_c^{(k)} \cdot x_{ij}^{(c)}$$

Применения:

- Изменение числа каналов (bottleneck)
- Снижение размерности перед 3×3 свёрткой
- Аналог FC-слоя для каждого пикселя

Bottleneck в ResNet



3×3 свёртка работает **в 4 раза быстрее** (64 канала вместо 256).

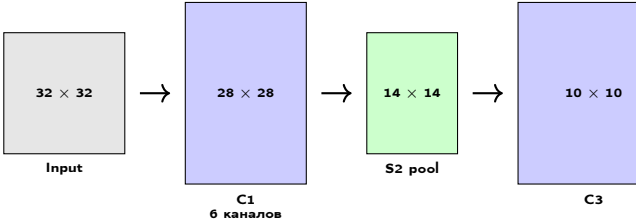
LeNet-5 (LeCun, 1998)

Первая успешная CNN. Для распознавания рукописных цифр (MNIST).

Архитектура:

- Вход: 32×32 grayscale
- Conv $5 \times 5 \to 28 \times 28 \times 6 \to$ AvgPool $2 \times 2 \to 14 \times 14 \times 6$
- Conv $5 \times 5 \to 10 \times 10 \times 16 \to$ AvgPool $2 \times 2 \to 5 \times 5 \times 16$
- Conv $5 \times 5 \to 1 \times 1 \times 120$ (FC)
- FC 84 $\to$ FC 10 (softmax)

Итог: 60K параметров.



Наследие: pattern свёртка $\to$ пулинг $\to$ свёртка $\to$ пулинг $\to$ FC стал шаблоном для всех CNN.

AlexNet (Krizhevsky, 2012)

Победитель ImageNet 2012 — переломный момент для DL.

Ключевые инновации:

- ReLU вместо tanh
- Data augmentation (сдвиги, отражения)
- Dropout (0.5 на FC-слоях)
- Local Response Normalization
- Обучение на 2 GPU параллельно

Архитектура:

- 5 свёрточных + 3 FC слоя
- Conv: 11×11 (96 каналов), 5×5 (256), 3×3 (384, 384, 256)
- MaxPool 3×3 после первых двух и последней свёртки
- FC: $4096 \to 4096 \to 1000$ (softmax)

Детали

Параметр	Значение
Параметры	62 млн
Размер входа	$224 \times 224 \times 3$
Batch size	128
Learning rate	10^{-2}, decay $\times 10$
Эпох	90
Время	5–6 дней

Влияние:

- Показал, что DL работает на реальных задачах
- Запустил эру глубокого обучения
- Все современные CNN — наследники AlexNet

VGG (Simonyan & Zisserman, 2014)

Главная идея: стек из 3×3 свёрток.

Два 3×3 слоя $\rightarrow$ RF 5×5. Три 3×3 слоя $\rightarrow$ RF 7×7.

Почему это выгодно?

- 3×3: 9 параметров на канал
- 5×5: 25 параметров на канал
- 7×7: 49 параметров на канал
- + Дополнительные нелинейности (ReLU)

VGG:

- Conv 3×3: 64 $\rightarrow$ 128 $\rightarrow$ 256 $\rightarrow$ 512 $\rightarrow$ 512
- MaxPool 2×2 после каждой группы
- 3 FC слоя (4096, 4096, 1000)

Сравнение

	VGG16	VGG19
Параметры	138M	144M
Слоёв (weights)	16	19
ImageNet top-5	8.8%	7.5%
Размер модели	528 MB	548 MB

Недостаток: очень много параметров (138M), тяжело.

Влияние: показал, что **глубина** важнее ширины. Унифицированный дизайн стал стандартом.

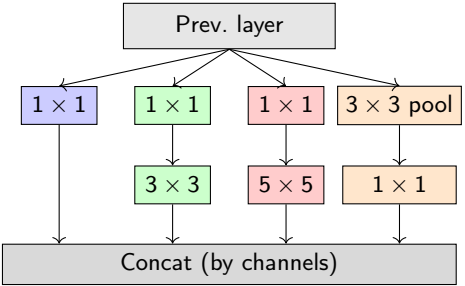
Inception (GoogLeNet, Szegedy 2014)

Проблема: какой размер ядра выбрать? 1×1, 3×3, 5×5?
Решение: используем **все сразу!**

Inception module:

- Параллельные ветки:
  - 1×1 свёртка
  - $1 \times 1 \to 3 \times 3$
  - $1 \times 1 \to 5 \times 5$
  - 3×3 MaxPool $\to 1 \times 1$
- Конкатенация по каналам

Bottleneck: 1×1 перед дорогими 3×3 и 5×5 снижает размерность.



6.8M параметров (VGG: 138M)!

ResNet (He et al., 2015)

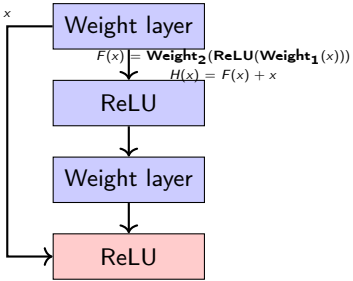
Проблема: при > 20 слоях качество падает (degradation problem).
Не из-за overfitting: и train, и test loss растут.

Решение: Residual Learning

$$F(x) = H(x) - x \quad \Rightarrow \quad H(x) = F(x) + x$$

- Слой учится **добавке** к тождественному отображению
- Если слой не нужен: $F(x) \to 0$, $H(x) \to x$
- Градиент течёт через shortcut напрямую

Результат: 152 слоя, 3.57% top-5 error на ImageNet.



Bottleneck block:
1×1 $(256 \to 64) \to 3 \times 3$ $(64 \to 64) \to$
1×1 $(64 \to 256)$

DenseNet (Huang et al., 2017)

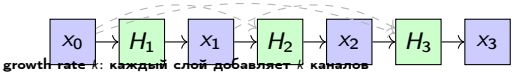
Идея: каждый слой получает на вход **feature maps со всех предыдущих слоёв**.

$$x_l = H_l([x_0, x_1, \ldots, x_{l-1}])$$

Почему это работает:

- Максимальный поток информации и градиентов
- Каждый слой имеет прямой доступ к градиенту лосса
- Естественная регуляризация
- Меньше параметров (нет дублирующих признаков)

Недостаток: большой расход памяти (нужно хранить все активации).



DenseNet-121: 8M params vs ResNet-50: 25M (при схожем качестве).

MobileNet (2017) и EfficientNet (2019)

MobileNet:

- Depthwise Separable Conv
- Width multiplier (α)
- Resolution multiplier (ρ)
- 4.2M params, 569M FLOPs

MobileNetV2:

- Inverted Residual: узкий $\rightarrow$ широкий $\rightarrow$ узкий
- Linear bottleneck (ReLU не убивает информацию в узком слое)

EfficientNet (Tan & Le, 2019):
Ключевая идея: композитное масштабирование

Обычно: масштабируют **один** параметр (глубину, ширину или разрешение).
EfficientNet: масштабирует **все три** сбалансированно:

$$d = \alpha^\phi$$

$$w = \beta^\phi$$

$$r = \gamma^\phi$$

где $\alpha\beta^2\gamma^2 \approx 2$ (FLOPs растут в 2^ϕ раз).

EfficientNet-B7: 66M params, достигает SOTA при **в 10x меньше параметров**, чем предыдущие SOTA.

Сводка архитектур

Архитектура	Год	Парам.	Top-5	Ключевая идея
LeNet-5	1998	60K	—	Первая CNN
AlexNet	2012	62M	16.4%	ReLU, Dropout, GPU
VGG16	2014	138M	8.8%	3×3 стеки
GoogLeNet	2014	6.8M	6.7%	Inception modules
ResNet-152	2015	60M	3.6%	Skip connections
DenseNet-121	2017	8M	5.6%	Dense connections
MobileNetV2	2018	3.5M	—	Depthwise sep.
EfficientNet-B7	2019	66M	2.2%	Compound scaling

Правило большого пальца

Чем новее архитектура — тем больше качество при меньшем числе параметров. Но ResNet — «рабочая лошадка» до сих пор.

Заключение

Что мы узнали:

- Свёртка — эффективная альтернатива полносвязным слоям
- Свёрточная арифметика: I, K, P, S
- Базовые блоки: свёртка, пулинг, транспонированная свёртка
- Специализированные: dilated, depthwise separable

Эволюция архитектур:

- LeNet: паттерн
- AlexNet: масштаб
- VGG: глубина
- Inception: ширина
- ResNet: skip connections
- DenseNet: dense connections
- EfficientNet: compound scaling

«В глубоком обучении глубина — это новая ширина»

— но, как показал Transformer, иногда внимание важнее глубины